AutoSaaS AI

Complete Code Documentation & Judge Q&A Guide

A deep-dive into every file, function, syntax, and design decision. Written so you can confidently answer any judge question.

Tech Stack: FastAPI + SQLite + Playwright + Browserbase + Tailwind CSS

Files: 6 source files | 25+ functions | 4 test cases (TC1-TC4)

Table of Contents

- 1. Project Overview
- 2. Architecture Diagram
- 3. Data Flow
- 4. Database (database.py)
- 5. API Server (main.py)
- 6. Agent (agent.py)
- 7. Frontend (index.html)
- 8. Mock Console (mock_console.html)
- 9. Dependencies
- 10. Test Cases TC1-TC4
- 11. Performance
- 12. Judge Q&A
- 13. Glossary

1. Project Overview & Architecture

AutoSaaS AI is an autonomous user provisioning system. Takes CSV of users (email+role), validates/dedups, then uses browser automation agent to create each user in a SaaS admin console. Progress tracked in SQLite, shown live on web dashboard.

Problem Solved

Manually creating 500 users in a SaaS admin console takes hours. This tool automates it with a browser agent, handles errors, supports retries, provides audit trails.

Key Features

- CSV upload with email validation (TC1) and dedup (TC2)
- Role normalization: CSV roles mapped to Admin/Manager/Member (TC3)
- Idempotent retries: only re-process failed/pending (TC4)
- Human-in-the-Loop approval gate
- Live progress bar polling SQLite every 500ms
- Audit logging for compliance
- Local (Chromium) and cloud (Browserbase) browser support

2. System Architecture Diagram

```
FRONTEND (Browser)
  index.html (Dashboard)  +  mock_console.html (Target SaaS Console)
  | HTTP (fetch) |
  v
BACKEND (FastAPI) on port 8000
  main.py: /api/upload-csv, /api/approve-run, /api/run-progress, /api/run-agent
  agent.py: BrowserbaseProvisioningAgent
  database.py: SQLite (runs, records, audit_logs)
```

3. Data Flow Diagram

```
USER uploads CSV
  |
  v
[POST /api/upload-csv]
  +-- Parse CSV with pandas
  +-- Validate columns (email, role)
  +-- TC1: Validate email regex
  +-- Normalize: email.strip().lower(), role.strip()
  +-- TC2: Drop duplicates by email (keep first)
  +-- Assign IDs (REC-001, REC-002, ...)
  +-- save_parsed_csv() -> SQLite (status='pending')
  |
  v
Frontend shows Review Table (user approves)
  |
  v
[POST /api/approve-run] -> status='approved'
  |
  v
[POST /api/run-agent]
  +-- Check run exists in DB
  +-- Check non-success records exist (TC4)
  +-- Enforce approval gate
  +-- BrowserbaseProvisioningAgent.run()
  |
  v
Agent loops over pending records:
  +-- Normalize role (admin->Admin, etc.)
  +-- page.evaluate(JS): open modal, fill, submit
  +-- Read last table row -> verify email+role
  +-- update_record_status() -> 'success' or 'failed'
  +-- log_audit_event()
  |
  v
Frontend polls GET /api/run-progress every 500ms
  +-- completed = COUNT(status='success')
  +-- total = runs.total_records
  +-- percentage = completed/total * 100
  |
  v
All done -> status='completed' -> Show results table
```

4. Database Schema (database.py)

File: backend/database.py | 114 lines | SQLite3 + pandas

Why SQLite?

Zero-config, file-based, no separate server. Perfect for single-machine provisioning. DB file persists between runs, enabling retries.

Table 1: runs

```
CREATE TABLE runs (run_id TEXT PRIMARY KEY, filename TEXT, total_records INTEGER, status TEXT)
```

Table 2: records

```
CREATE TABLE records (id TEXT, run_id TEXT, email TEXT, role TEXT, status TEXT, error TEXT, PRIMARY KEY
```

Table 3: audit_logs

```
CREATE TABLE audit_logs (log_id INTEGER PRIMARY KEY AUTOINCREMENT, run_id TEXT, timestamp DATETIME DEFA
```

Function: init_db()

Creates all 3 tables if they don't exist. Safe to call repeatedly (IF NOT EXISTS). Called on app startup and before save_parsed_csv.

Function: save_parsed_csv(run_id, filename, records)

Inserts run row + all record rows. Uses INSERT OR REPLACE for idempotency. Status starts as 'pending'.

Function: get_pending_records(run_id)

SELECT * FROM records WHERE run_id=? AND status != 'success'. Uses pandas read_sql_query. Returns list of dicts. This is the TC4 retry mechanism - already-successful records are skipped.

Function: update_record_status(run_id, record_id, status, error)

UPDATE records SET status=?, error=? WHERE run_id=? AND id=?. Called by agent after each record. This is what drives the live progress bar.

Function: update_run_status(run_id, status)

UPDATE runs SET status=? WHERE run_id=?. Tracks overall run state.

Function: log_audit_event(run_id, event_type, details)

7. Frontend Dashboard (index.html)

File: frontend/index.html | ~660 lines | Tailwind CSS + Vanilla JS

- Header with title + Agent Active badge
- Step progress tracker (Step 1 of 3)
- Loading panel (shown during CSV validation)
- Upload section: file input + Upload & Validate button
- Review table: parsed records with status
- Approve & Run Agent button
- Execution status panel with progress %
- Results table with CSV role vs applied role

approveAndRun() - Live Progress Polling

```
  const pollInterval = setInterval(async () => { fetch /api/run-progress every 500ms; update progress % }
```

Polls /api/run-progress every 500ms. Updates progress % in real-time based on DB counts. Cleared on completion or error.

renderResultsTable(results, elapsed)

Builds HTML table. Shows: #, Email, CSV Role, Applied Role (with TC3 normalization badge), Status badge, Verification. Highlights role changes.

8. Mock SaaS Console (mock_console.html)

File: frontend/mock_console.html | 122 lines | Simulated admin console
This is the automation TARGET. The agent navigates here and creates users.

Key DOM Elements

```
  button#open-modal-btn, input#email-input, select#role-dropdown (defaults to Admin - the bug), button#su
```

Why 'Default Bug'?

The role dropdown defaults to 'Admin' regardless of what the agent sets. This simulates a real SaaS bug (TC3). The agent must explicitly set the role value before submitting. If it doesn't, all users become Admin - a security issue.

9. Dependencies (requirements.txt)

10. Test Cases TC1-TC4 Explained

TC1: Email Validation

Requirement: Reject invalid email formats during CSV upload. Implementation: Regex pattern in main.py upload_csv. If any email fails, returns HTTP 400. Why: Prevents garbage data. Fail-fast.

TC2: Deduplication

Requirement: Remove duplicate emails, keep first occurrence. Implementation: df.drop_duplicates(subset=['email'], keep='first'). Also normalizes emails to lowercase first. Why: Case-insensitive dedup.

TC3: Role Normalization

Requirement: Map CSV roles to canonical Admin/Manager/Member, correcting the mock console's default 'Admin' bug. Implementation: role_map dict in agent.py. Default 'Member'. Agent explicitly sets dropdown value. Why: Security.

TC4: Idempotent Retries

Requirement: Re-running agent only processes failed/pending records, skips successful ones. Implementation: get_pending_records() filters status != 'success'. run-agent checks retry_needed_count. Why: Saves time, avoids duplicates.

11. Performance Optimizations

```
BEFORE (slow):
  - 4 Playwright calls per record
  - 0.5s sleep between operations
  - wait_for_selector before each action
  - O(n^2) verification (scan all rows)
  - ~2-3 seconds per record

AFTER (fast):
  - 1 page.evaluate() JS call per record
  - No sleep, no wait_for_selector
  - O(1) verification (read last row only)
  - ~0.1-0.2 seconds per record
  - ~10-20x speedup
```

Optimization 1: Single JS Round-Trip

Playwright calls have IPC overhead (~50-100ms each). Combining 4 calls into 1 page.evaluate() eliminates 3 round-trips.

Optimization 2: Remove Sleeps

Original code had time.sleep(0.5) and wait_for_selector. Since mock console is synchronous, no wait needed.

Optimization 3: Targeted Verification

Original: read entire table. New: read only last row (tr:last-child). O(1) instead of O(n).

12. Judge Q&A - Likely Questions & Answers

Q: How does the progress bar work?

A: Frontend polls /api/run-progress every 500ms. Backend counts records with status='success' in SQLite and returns {completed, total}. Percentage = completed/total*100.

Q: How do you handle errors?

A: Per-record try/except in agent.py. Failed records get status='failed' with error message. Batch continues. Failed records can be retried via TC4.

Q: How is security handled?

A: Role normalization (TC3) prevents privilege escalation. Email validation (TC1) prevents injection. CORS configured. Audit logs for compliance.

Q: Can it scale to 500+ users?

A: Yes. Optimizations make it ~10-20x faster. SQLite handles concurrent reads. Progress polling is lightweight.

Q: What if the agent crashes mid-run?

A: Records already processed have status='success' in DB. Re-running only processes remaining 'pending'/'failed' records (TC4). Idempotent.

Q: How do you test the role normalization?

A: Mock console defaults role to 'Admin' (bug). Agent explicitly sets dropdown value. Verification checks last row contains correct role.

Q: What is Human-in-the-Loop?

A: After CSV upload, user reviews parsed records in dashboard. Must click 'Approve & Run Agent' before automation starts.

Q: Why Playwright over Selenium?

A: Playwright has faster execution, better API, auto-wait, and Browserbase cloud support.

Q: How does Browserbase work?

A: Cloud browser service. Agent connects via CDP. Session URL provided for live viewing. Falls back to local Chromium for dev.

Q: What database do you use and why?

A: SQLite. Zero-config, file-based, no separate server. Persists between runs for retry support.

13. Glossary of Key Terms

CDP	Chrome DevTools Protocol - used to connect Playwright to Browserbase cloud browser
CORS	Cross-Origin Resource Sharing - allows frontend to call backend API
HitL	Human-in-the-Loop - approval gate before automated actions
Idempotent	Safe to run multiple times - already-done work is skipped
IPC	Inter-Process Communication - overhead of Playwright calls
Playwright	Browser automation library by Microsoft
SQLite	File-based relational database, zero configuration
TC1-TC4	Test Cases 1-4: Email validation, Dedup, Role normalization, Retries
FastAPI	Modern Python web framework with async support
pandas	Python data analysis library for CSV processing
repr()	Python function returning string representation with proper quoting
O(1) vs O(n)	Constant time vs linear time - performance complexity